

Movie Recommendation System Using Collaborative Filtering

Abstract

This study explores the design and implementation of a movie recommendation system using collaborative filtering techniques. Movies can be recommended in two primary ways. A list can be generated based on the last movie a user watches or by the user's rating history. In the study, various collaborative filtering algorithms are explored and compared against a dataset containing 45,115 movie titles and 270,896 users. These algorithms include both user and item based nearest neighbor algorithms, and the singular value decomposition (SVD) model. Exploration is conducted into designing a system which can reliably produce personalized recommendations of unseen movies to a user. A SVD model was successfully implemented and could predict a confident list of movie recommendations for a target user or movie.

1 Introduction

Recommendation systems commonly work by attempting to predict a user's rating of an unknown item. The system looks for similarities between objects or users in many different applications such as movies, news articles, books, music, and general goods (Mohamad, 2022).

Recommendation systems have become vital for many large companies who rely on user attention. Netflix, Spotify, Amazon, and Meta all implement ways to recommend media, products, or services to users (Hsieh, 2017).

Netflix and Amazon Prime Video relate the closest to this study with how they recommend movies to users. Netflix uses a sophisticated recommendation system which in addition to using a collaborative filter, uses the movie content (genre, lead actors, directors) for insight and user driven data. User data includes implicit ratings of a film or tv show and the users viewing habits (Netflix, 2025). Implicit ratings are created from a series of metrics which allows Netflix to determine a viewer's interest without direct ratings. Usually this is done with a medias "like" rating prompt after the film, how long the user watched the content, and their watch history.

In addition, user habits also allow Netflix to recommend different content based on which device is used and the time of day. For example, if cartoons are watched on a tablet during the day, it can be assumed a child is watching. Netflix can then recommend cartoons during the day on a tablet for the child and on the contrary, recommend long form media enjoyed by adults on a tv in the evening.

2 Data Description

The data used is from the website Kaggle named “The Movies Dataset”. This dataset is referenced from the MovieLens dataset by GroupLens Research. The dataset consists of five main files. For collaborative filtering, the files this study requires is the “metadata.csv” for movie titles, the “ratings.csv” and “ratings_small.csv” for user movie ratings, and the “links.csv” and “links_small.csv” for tying the ratings movie ID to the metadata ID. The ratings file contains four numeric variables, and is comprised of a user ID, a movie ID, the users rating of the movie ID, and the timestamp of when the rating occurred. The dataset is noted to contain 45,000 movies, 270,000 users, and a total of 26 million ratings. There is also a small version of only 100,000 ratings available.

3 Background and Techniques

3.1 Recommender Considerations

3.1.1 Imbalanced Datasets

For recommendation systems, an imbalanced dataset refers to a situation where some item’s representation in the dataset is significantly more than others. For example, there are a set of highly rated popular blockbuster films in the dataset. Training models with this imbalance can lead to biases towards the majority. These biases cause the model to struggle when attempting to predict the minority class (Sivek, 2024). For a recommendation system, this means the recommendations are likely to be less

personalized or unique to the target user. Additionally, when the model learns noise and outliers in the training data, it can negatively impact the models’ performance on unseen data. The models’ high sensitivity to fluctuations in the data creates a large variance error and cause the model to be overfitted. On the contrary, if a model is too simplistic, it is unable to catch the overall trend of the data. It will then have a large bias error and be considered underfitted (Sivek, 2024).

3.1.2 The Cold Start Problem

The cold start problem in recommendation systems occurs when the system has little to no historical data points to determine inferences. New users will find their initial recommendations to be generic until the system is able to gather enough information to draw insights from it. This problem can also affect new movie releases since the new film will initially start with no user reviews for a period of time. Methods to handle this problem are discussed in detail further on in the paper.

3.2 Models

The collaborative and SVD models are supervised algorithms fitted to a labeled dataset. Each of the models listed have various strengths and weaknesses which are explored and tested.

3.2.1 kNN

The k-nearest neighbors (kNN) algorithm is a supervised learning classifier which uses nodes proximity (neighbors) from one another to predict an unknown nodes classification. The prediction is labeled

based on a majority vote premiss. There can be multiple categories the data point can be assigned to. As popular as kNN is, it is still considered a “lazy learning” model. The kNN model uses memory to store its training data and will require that all the computation occur when a prediction is made. Normalization methods can be used to increase its effectiveness (IBM, 2021).

$$d(x, y) = \sqrt{\sum_{i=1}^n (y_i - x_i)^2}$$

Euclidean distance

3.2.2 SVD

The singular value decomposition (SVD) is a matrix factorization technique popular in recommendation systems. It is a robust approach for predicting unknown ratings by reducing insignificant features. The SVD technique performs well on data which changes frequently such as video, movie, and music streams. It also can overcome sparse matrix's when data is abundant (Nissa, 2023).

$$A_{m \times n} = U \sum V^T$$

Real matrix A size $m \times n$. where U and V are orthogonal matrices of sizes $m \times m$ and $n \times n$ respectively

4 Design

The design of the recommendation system was modeled using three different techniques. Each technique was able to perform the recommendation task with varying degrees of success. These

include an item-base collaborative filter, a user-base collaborative filter, and the SVD technique. Within each model, this study experimented with different methods to improve the model's effectiveness.

4.1 Data Quality

To increase the success of the recommendation models, the quality of the data input must be free of duplicates, missing values, inconsistencies, irrelevant data, and errors (Rogers & Jonker, 2024). In the recommendation system design, as series of steps were taken to assure the data remains accurate and reliable.

4.1.1 Preprocessing

Data preprocessing is a strongly recommended step before the data analysis. Its goal is to transform the data into a format which is both easier and more effectively processed by the learning model created. Visual plots such as histograms, scatter plots, and heatmaps help aid each of these preprocessing techniques.

From the metadata file, all columns except for the **movieId** and **title** were dropped. The collaborative filter-based design will not use any of the other features. The counts of the two rows were 45,466 and 45,460 respectively. Upon inspection it was noted that there were rows where the titles were missing. It was also discovered that there were rows where the **movieId** was not a numeric number. Once the non-numeric **movieId** rows and empty titles were dropped, the data was checked for duplicates. The first instance of a duplicate **movieId** was kept

bringing the total movies in the metadata file from 45,466 to 45,430.

The ratings file had the same scrubbing methods applied. This resulted in the removal of approximately 8,000 duplicate user ratings. In many cases the duplicated ratings are usually averaged. However, since the file also provides the timestamp, the most recent rating was taken. An interesting metric to note is the number of ratings a user makes and the number of ratings a movie receives.

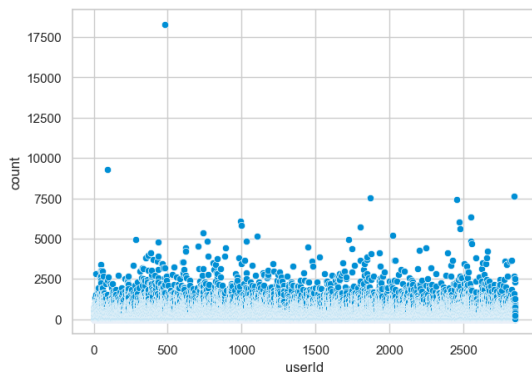


Figure 1: number of ratings per userId

Figure 1 and 2 display outliers which will affect the model training. A user who rated over 17,500 movies for example, will be overrepresented in the user-based collaborative filter and should be removed. Conversely the opposite is also true. Users with few ratings or movies with little ratings introduce noise in the system and should also be filtered out.

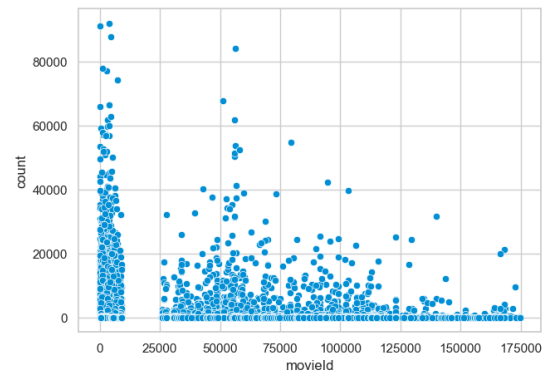


Figure 2: number of ratings per movieId

Normalizing the numerical values to a set range has been shown to increase a machine learning models' performance overall. Some models do not perform well during the learning process when numeric data are distributed differently (Google, 2024). The ratings provided are already normalized on a scale from 0.5 to 5.0.

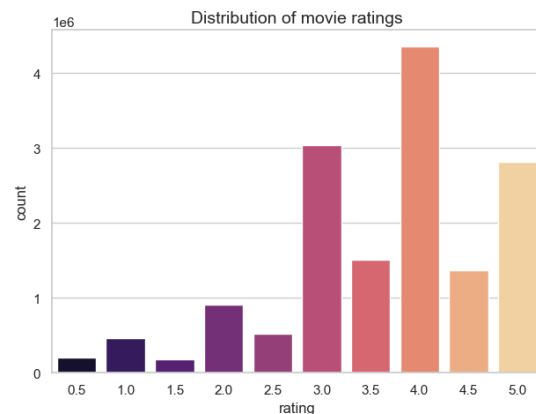


Figure 3: rating value distribution

However, it is beneficial to normalize a user's ratings within the user's own ratings. Figure 3 shows the distribution of movie ratings. The top three ratings a user submits is 4.0, 3.0, and 5.0 respectively. Whole numbers beat their half partners by a ratio of two to one. Lower ratings are also underrepresented in the dataset. This

is usually due to a few reasons. One is popularity bias where a small subset of items, attract a majority of interest and feedback (Gulsoy, 2025). Popular movies are popular for a reason, and it is likely the user it is recommended to will also like it. Poorly rated movies are less likely to be watched due to the negative stigma created from its average rating. The system also takes into account a user's rating habits. If a user tends to rate movies, they only dislike or like, it can introduce bias. Ratings for a user are based on their mean rating to account for this.

4.2 Model

For all three models tested, a matrix is first constructed between users and movies, with the cells as the movie's user rating. Due to the size of the dataset, the resulting matrix is sparse as not all users rate every movie. A sparse matrix is more memory efficient as the empty cells do not need to be filled with a value.

$$(userId \times movieId) = \begin{matrix} & \begin{matrix} 1 & n/a & 2 \end{matrix} \\ \begin{matrix} n/a \\ 3.5 \\ 5 \end{matrix} & \begin{matrix} n/a \\ 3.5 \\ n/a \end{matrix} \end{matrix}$$

4.2.1 User-based Collaborative Filter

A user-based collaborative filter recommends movies by relating the target user's preference to similar users. The target user is assumed to behave similarly as they have in the past. A $userId \times userId$ matrix is constructed to find similarities. By using k nearest neighbors, the system assigns a weight to each user. The weight represents the user's similarity with the target user. The model

selects n neighbors with the highest weights. Using those neighbors, the model then selects the best rated movies from a weighted average of those neighbors' favourite movies. A downside to this model is the amount of memory required to make the calculations. Movie watchers vastly outnumber the number of movies available and will create a large and sparse matrix. This will cause difficulty in finding meaningful patterns.

4.2.2 Item-based Collaborative filter

Like the user-based collaborative filter, the item-based filter works much in the same way. However, it focuses on the relationship between the movies by using a $movieId \times movieId$ matrix and recommends films similar to target users highest rated films. Using an item-based approach uses much less memory. However, the sparsity problem still exists.

4.2.3 Singular Value Decomposition

The singular value decomposition (SVD) is a powerful technique which recommendation systems use to reduce the dimensionality of the user-item matrix. It captures important latent factors which influence a user's preferences and the items features. The system then predicts the users perceived rating of any one movie. The system runs these predictions and returns movies with high predicted ratings.

4.3 Hypertuning

With each technique used to calculate a users rating, there exists the possibility to tune the model for better model performance. Hypertuning is the method of iteratively changing a model's specific

parameters in the pursuit of reducing error values. Parameters are selected from a set and are run on a 3 cross-fold validation of the model. The resulting scores and best parameters of the optimal model are returned for each model.

4.4 Recommending Films

As noted in the model sections, each prediction technique uses a different method to gather a list of movies it estimates target user will enjoy. The design of the system will reflect the best performing technique. To retrieve the movie title from the metadata file, the system must use the “links.csv” included. The **movieId** value in the ratings file does not directly translate to the id of the metadata file. The system creates a lookup table to map the ratings **movieId** to the metadata **movieId**. In the links file, the metadata **movieId** reference is denoted as the **tsmdbId** value.

5 Implementation

5.1 Environment

The environment this study used was a Jupyter notebook running python 3.12. The main libraries referenced include pandas, numpy, surprise, sklearn, and scipy. The relevant system specs of the machine include a Windows 11 operating system, an AMD Ryzen 9800X3D CPU and 32GB 6000MHz dual channel RAM.

5.2 Dataset & Preprocessing

The dataset used comes from the website Kaggle and is named The Movie Dataset. It is based on the MovieLens dataset from GroupLens Research. Two variations were used, the full 27 million ratings

dataset “ratings.csv”, and the 100k ratings dataset “ratings_small.csv”.

<https://www.kaggle.com/datasets/rounakbanik/the-movies-dataset/data?select=ratings.csv>

The data is scrubbed of errors and undergoes a series of filters to remove outliers and noise from the data. The recommender filters out the top 5% of movie raters and users who rate less than 10 movies. Movies with less than 50 ratings are also removed.

5.3 Data Split, Mapping and Matrix

The ratings data was split into a training and test dataframe using an 80-20 split with a global random seed of 42. The split was performed using the user’s timestamp to ensure each user is represented in both the training and test datasets.

User IDs and movie IDs are mapped to a consecutive integer indices for the matrix creation.

5.4 Models

Model	Hyperparameters	Fit Time 26m	Fit Time 100k
User-based kNN	K:60, minK: 2, calc:”msd”	Error	0.05s
Item-based kNN	K:60, minK: 1, calc:”msd”	35s	0.06s
SVD	n_factors:50, n_epochs: 15, lr:0.005	64s	0.12s

Table 1: Model overview

5.5 Metrics

Root mean square error (RMSE) and mean absolute error MAE are used to measure model performance.

5.6 Cold Start

Fall back to top 50 most popular movies the target user has not rated.

5.7 Source

https://github.com/liamdo12/CP_612_VA_1_recommendation_system/tree/master/submissions

6 Result & Analysis

6.1 Baseline

Each model was initially evaluated without any parameter changes using a 5 cross-fold validation. Each of the 5 splits evaluated the models performance using RMSE and MAE scores. The models average fit time was also noted. The data results as seen in *figure 4* show the SVD technique outperform the kNN item-based collaborative filter.

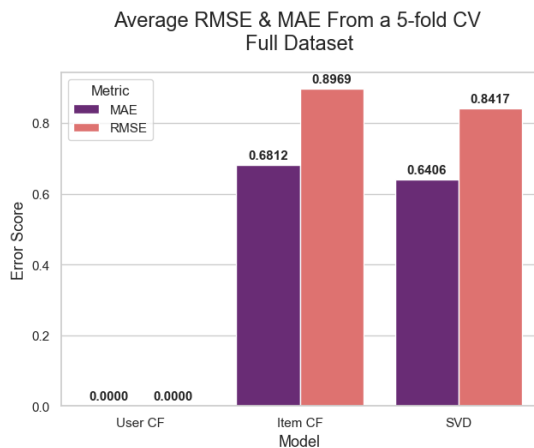


Figure 4: Average model performance computed from the full dataset

Unfortunately, the kNN user-based collaborative filter was unable to be processed due to memory constraints. During the similarity computation step, the system attempts to reserve memory for a $270k \times 270k$ matrix of float32 data.

The total allocated memory of this matrix is well above the test systems 32GB available memory. The rest of the analysis will use the smaller dataset for comparison. The results found here are still important as the SVD performed very well when there is more data available. Applying the baseline to the smaller dataset shows similar performance across all three models with the SVD continuing to hold better RMSE and MAE scores.

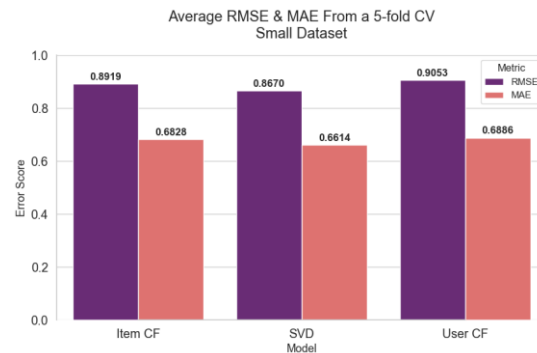


Figure 5: Average model performance using the small dataset

6.2 User Mean Adjustment

When adjusting for the users rating score habits, both collaborative filters improve their results. The item-based collaborative filter now slightly outperforms the SVD model, while the user-based filter matches the SVD model.

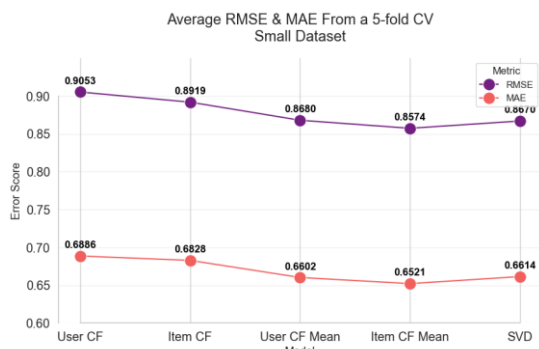


Figure 6: Model performance on the small dataset including kNN means

With the item-based collaborative filter performing well using user means, the results on the full dataset were again tested. The resulting scores did not change in a meaningful way with the introduction of more data compared to the SVD model. In figure 7 SVD is shown to improves its RMSE and MAE scores on the larger dataset.

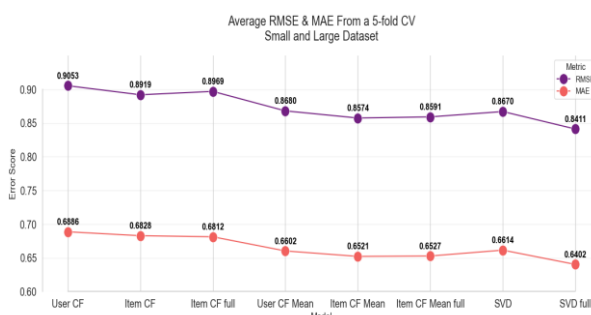


Figure 7: Average model performance displaying values for both the full and small datasets

The time impact of the different models depends on the action performed. On the small dataset, the collaborative filters displayed better fit times than the SVD by a factor of roughly 2.5. However, the SVD model performed much better in the test time where the best collaborative filter to over 10 times longer.

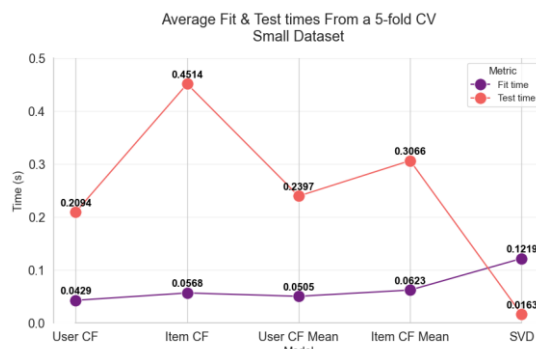


Figure 8: Average fit and test time in seconds on the small dataset

The full dataset exposes a weakness in the mean item-based collaborative filter where the test time almost doubles from the baseline according to the findings in figure 9.

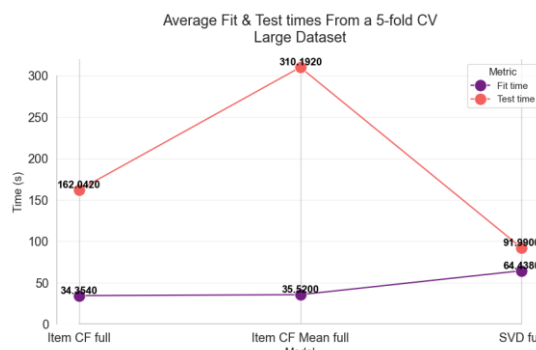


Figure 9: Average fit and test time in seconds on the full dataset

6.3 Hypertuning

Hypertuning was applied to each model in the pursuit of lowering error scores. Each model was given a set of parameters to apply. Each model applied a 3 cross-fold validation on every combination of the parameter set. Hypertuning is a time intensive task depending on the number of tunable variables and the number of combinations tested. The results show minimal impact in the collaborative filters

and a small improvement in the SVD model.

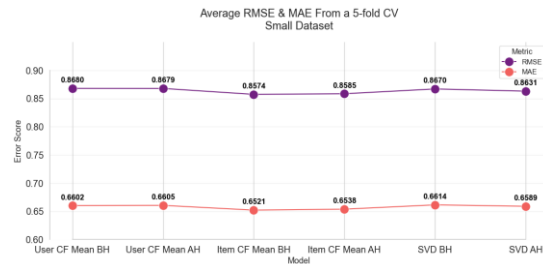


Figure 10: Average model performance compared to the hypertuned models

The best performing variables for the SVD model was setting the number of n_epochs to 15, the number for n_factors to 50, and the learning rate to 0.005.

Figure 11 below shows the effect on the RMSE for the variable used.

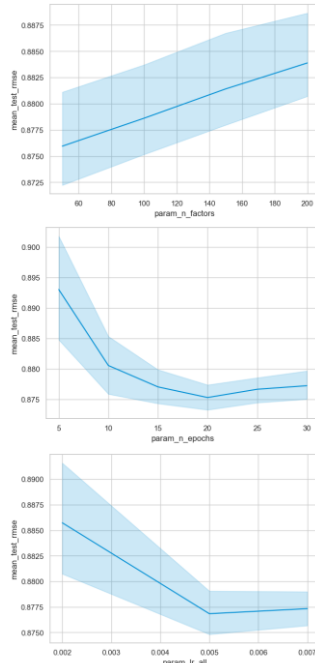


Figure 11: Examples of the SVD parameter effects

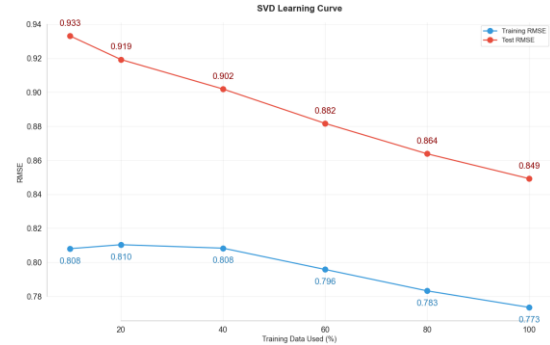


Figure 12: SVD learning curve

The above results affirm using an SVD model is the best choice for this study's movie recommender system.

Movie Title	Calculated Rating
The Shawshank Redemption	4.4466
The African Queen	4.4439
Cinema Paradiso	4.3815
The Godfather	4.3265
The Godfather: Part II	4.3176
Butch Cassidy and the Sundance Kid	4.2947
Amadeus	4.2895

Table 2: User 2 top 8 recommended movies and their predicted user rating

7 Improvement Opportunities

7.1 The Cold Start Problem

Warm start scenarios represent a best case where predictions are made on an item which is already represented in the database. A cold start happens when the user is new to the system, or a new movie is released.

The recommender system will struggle to recommend a new movie release to users

as there will be no initial user reviews on the movie. New users face the same problem as the system will struggle to make relevant recommendations if it cannot determine the new user's likeness to others.

For both scenarios, the model can be improved by using a hybrid content-based approach. The hybrid model would follow similarly to Netflix by computing a film sets feature similarities. These can include genres, lead actors, directors, keywords, and year released. A newly released movie can also be added to the dataset using critics ratings as reviewers generally receive early access to upcoming films to prepare their day one reviews.

On the user's side of the cold start problem, a common practice for companies like Netflix is to request the user to pick a few movies they like upon joining the service. While not explicit, this gives the recommendation system enough of a starting point to begin providing recommendations to the new user.

7.2 User Implicit Interactions

In the cases where users do not tend to rate their experiences, the system can infer a users experience through their habits. Most streaming services keep track of a user's interaction with the service. In terms of movie streaming, the system will likely record a users watch time and their watch history. With this information a recommendation system can infer a users rating. If a user does not finish a show or film or if the user

watches movies with a particular set of features such as genre, then the system can assign an implicit rating to the film even if it is left unrated by the user.

8 Conclusion

This study showed, given a set of user movie ratings, a recommendation system can be created to provide personal recommendations to a target user based on their past. Multiple matrix filtering techniques were tested resulting in competitive prediction results on the test data after hyperparameter tuning. The SVD matrix factorization model achieved the best RMSE and MAE scores on the large dataset at 0.84 and 0.64. While the collaborative filters performed similarly to the SVD on the smaller 100k dataset, the full dataset saw the collaborative filters struggle with the data size. The user-based collaborative filter was unable to allocate enough memory on the test system and the item-based saw little improvement.

9 References

- Mohamad, A. Y., Harun, S. R., Shahidan, N. A. A., Nanthamornphong, A., Mustapha, A., & Wahab, M. H. A. (2022). Collaborative Filtering Approach for Movie Recommendations. *2022 19th International Conference on Electrical Engineering/Electronics, Computer, Telecommunications and Information Technology (ECTI-CON)*, 1–6. <https://doi.org/10.1109/ECTI-CON54298.2022.9795373>
- Hsieh, M., Chou, W., & Li, K. (2017). Building a mobile movie recommendation service by user rating and APP usage with linked data on Hadoop. *Multimedia Tools and Applications*, 76(3), 3383-3401. <https://doi.org/10.1007/s11042-016-3833-0>
<https://help.netflix.com/en/node/100639>
- IBM. (2021, October 4). What is the k-nearest neighbors (KNN) algorithm? [Ibm.com. https://www.ibm.com/think/topics/knn](https://www.ibm.com/think/topics/knn)
- Nissa, F., Primandari, A. H., & Thalib, A. K. (2023). Collaborative Filtering Approach: Skincare Product Recommendation Using Singular Value Decomposition (SVD). *Media Statistika (Online)*, 15(2), 139–150. <https://doi.org/10.14710/medstat.15.2.139-150>
- Rogers, J., & Jonker, A. (2024, November 29). Data Cleaning. [Ibm.com. https://www.ibm.com/think/topics/data-cleaning](https://www.ibm.com/think/topics/data-cleaning)
- Gulsoy, M., Yalcin, E., & Bilge, A. (2025). EquiRate : balanced rating injection approach for popularity bias mitigation in recommender systems. *PeerJ. Computer Science*, 11, Article e3055. <https://doi.org/10.7717/peerj-cs.3055>
- Google. (2024). *Numerical Data: Normalization | machine learning | google for developers*. Google. <https://developers.google.com/machine-learning/crash-course/numerical-data/normalization>
- Sivek, S.C. (2024) *Mastering binary classification: A powerful predictive analytics tool*, Pecan AI. Available at: <https://www.pecan.ai/blog/mastering-binary-classification-model-predictive-analytics/> (Accessed: 04 December 2024).
- Netflix. (2025). How Netflix's Recommendations System Works. Netflix Help Center. Retrieved November 23, 2025, from <https://help.netflix.com/en/node/100639>